CSC3041F AI: Automated Plan & Control



Zola Mahlaza

Department of Computer Science

Feb 2026

Course contacts

Lecturer emails:

- Zola Mahlaza (Part 1): zmahlaza@cs.uct.ac.za
- Asad Jeewa (Part 2): Asad.Jeewa@uct.ac.za
- Krupa Prag (Part 3): krupa.prag@uct.ac.za

TA email:

- Ben Ruijsch Van Dugteren (RJSBEN001@myuct.ac.za)

Intro to state space search



Zola Mahlaza

Department of Computer Science

Feb 2026

Topics

- Overview of state space search in AI
- Core concepts: states, actions, transitions, and costs

Core task

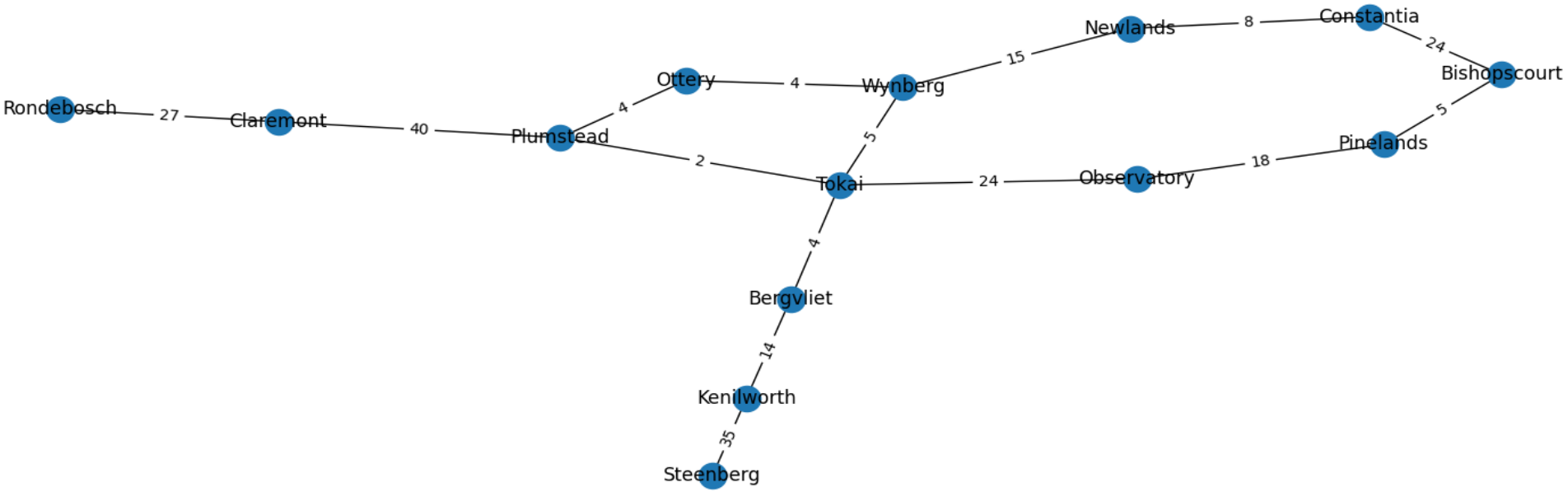
- Search: ***process of finding a solution, data item, path, or pattern within a defined computational space that satisfies a specific query***

AI Search

- Search is a universal problem-solving mechanism in AI
- In AI, sequence of steps for solving problem is not known a priori
  - trial-and-error exploration
- Three kinds of problems:
  - *single-agent path finding problems*
  - *game playing*
  - *constraint-satisfaction problems*

Example problem

- Imagine you are creating an autonoumous agent to deliver packages between two points
  - Consider delivering from Tokai to Rondebosch ...
- When environment is observable, what should agent do?



Problem solving framework

- Problem-solving agent framework
- Four-phase: goal formulation, problem formulation, search, and execution

Problem solving framework

- **Goal**: e.g. *Rondebosch*
  - Limit the objectives and actions to be considered.
- **Problem formulation**: What states and actions are necessary to reach the goal?
- **Search**: simulate sequences of actions until you have actions that reaches goal
- **Execution**: The agent can now execute the actions in the solution, one at a time

Problem solving framework

- Environment: fully observable, deterministic, known environment:
  - solution is a fixed execution of actions
  - Percepts can be ignored (open-loop system)
- If environment is nondeterministic, monitor the percepts (closed-loop approach)

Problem solving framework

What mathematical structure is required?

- **Goal**: e.g. *Rondebosch*
  - Limit the objectives and actions to be considered.
- **Problem formulation**: What states and actions are necessary to reach the goal?
- **Search**: simulate sequences of actions until you have actions that reaches goal
- **Execution**: The agent can now execute the actions in the solution, one at a time

Problem spaces and instances

- Problem space: every possible situation that can be encountered
- Problem instance: a specific, concrete version of that problem that needs a solution





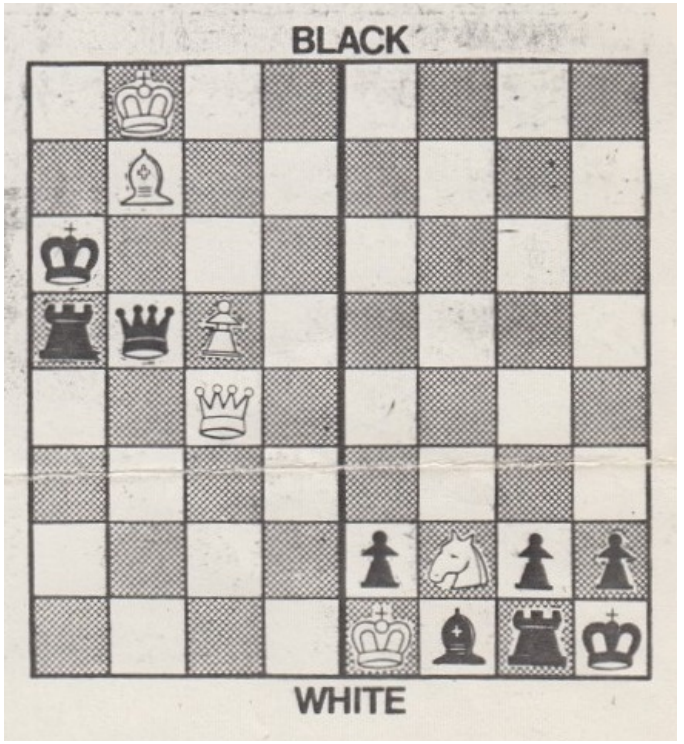
Image src: Wikipedia

Problem spaces

- A problem space is a set consisting of all possible states in a ‘game’, governed by the application of a set of allowed actions and relationship between them
- Terminology
  - **State space**
  - **Actions**
  - **Transition model**

Problem spaces: state space

- A state is a specific configuration of the problem
- Examples: positions in a maze, game boards
- A state $s \in S$, where S is a state space, is set of n features or variables $s=(v_1,v_2,v_3, \ldots, v_n)$
- State space as a graph
  - Nodes



Src: Gordon Pollard, Abingdon Herald, August 1978

Problem spaces: actions

- Actions are the possible moves or changes
- Actions (will) impact state transitions
- Examples: moving in a grid, making a move in a game
- An action a is a function that maps one state to another. Often, you have limited # of actions
- Example:
  - Let s = {(Nw:a1),(Kw:g1),(Kb :g8),Turn: White}
  - A(s) = {N → b3, N → c2, K → f1, K → f2, K → h1, K → h2}

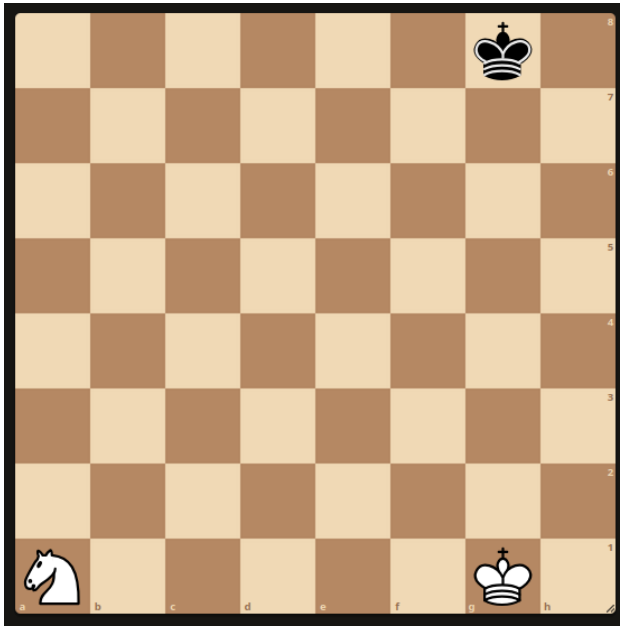


Image created via https://lichess.org/editor

Problem spaces: transitions

- Transition function models how a state transitions to other states
- Examples: impact of (likely) move to in a game
- Types: deterministic or includes uncertainty (i.e., probabilistic/stochastic)
- A transition model is a function $T : S \times A \rightarrow S$
- Example:
  - Let s = {(Nw:a1),(Kw:g1),(Kb:g8),Turn: White}
  - Consider $A(s)=\{N \rightarrow b3, N \rightarrow c2, K \rightarrow f1, K \rightarrow f2, K \rightarrow h1, K \rightarrow h2\}$
  - $T(s, N \rightarrow b3)$ = {(Nw:b3),(Kw:g1),(Kb :g8),Turn: Black}

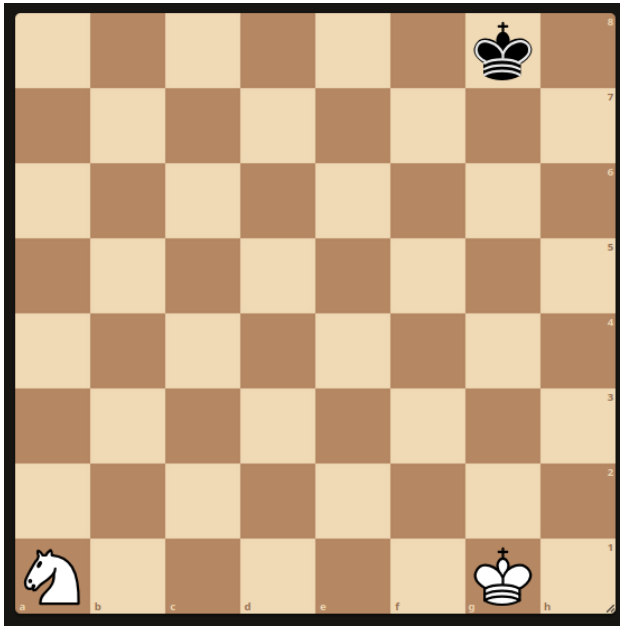
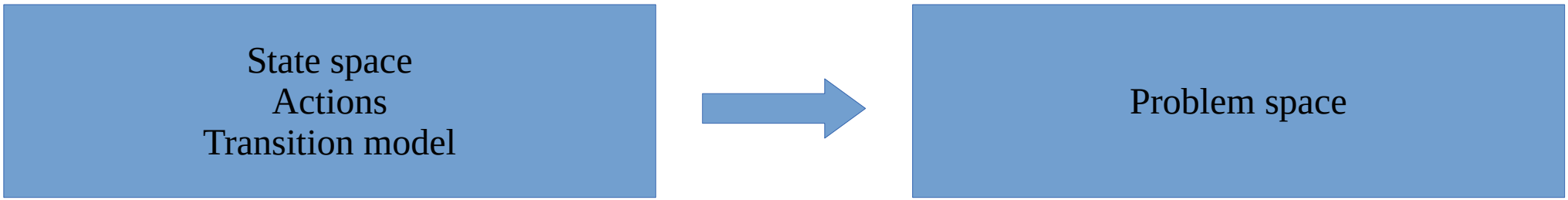


Image created via https://lichess.org/editor

Problem spaces

- A problem space consists of a set of states of the problem, and a set of operators that change the state



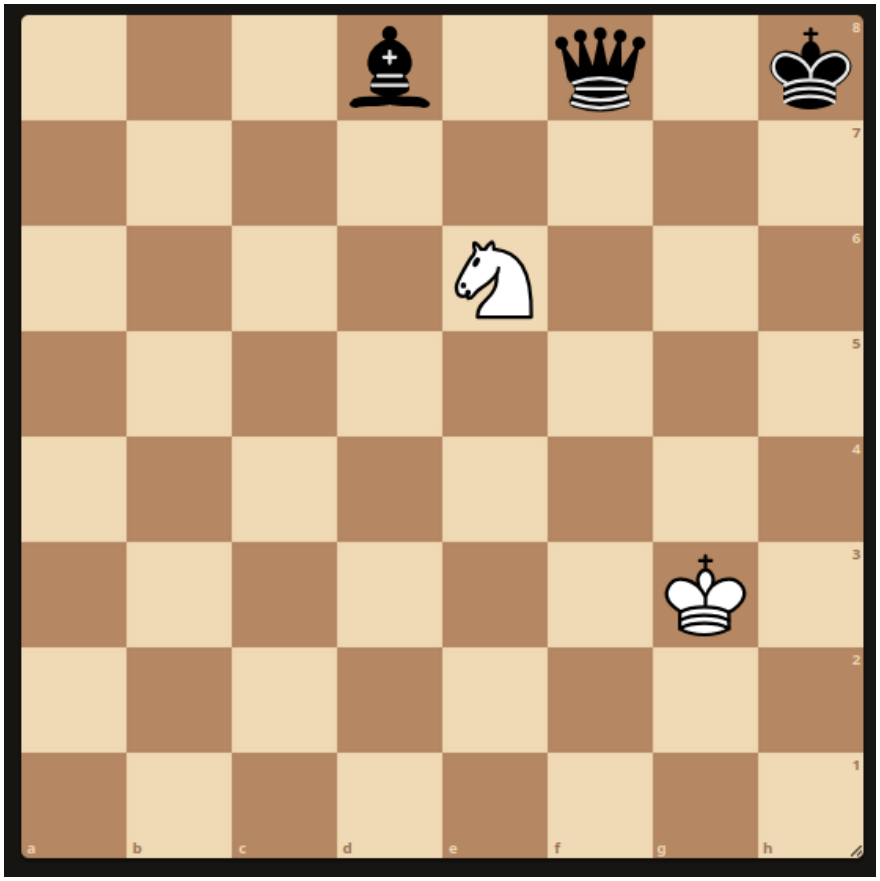
Problem instances

- Terminology/notions
  - **Action cost model**
  - **Initial state**
  - **Goal state(s)**

Problem instances: action cost

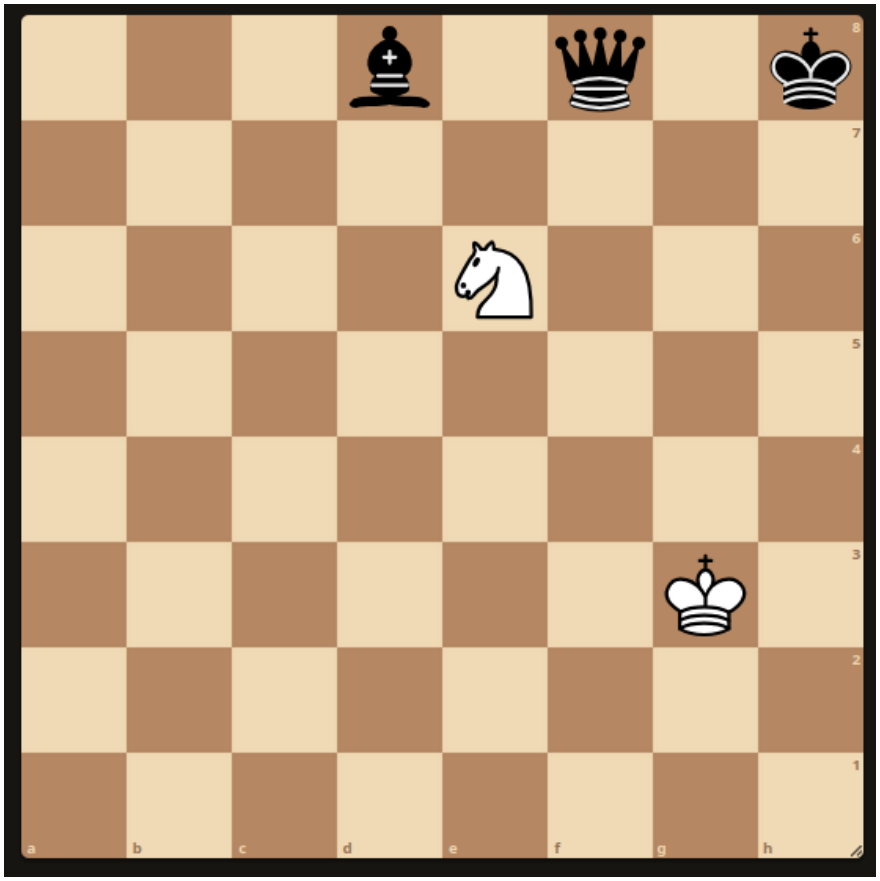
- Costs associated with moving from one state to another
- Examples: time, energy, or resource consumption
- Impact on search efficiency

Problem instances: action cost



- If the cost of every move is 1, is moving the Bishop to safety equivalent to moving Queen?

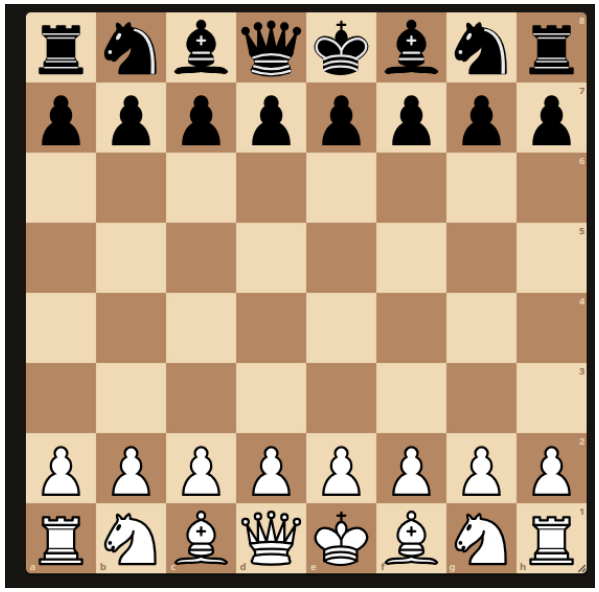
Problem instances: action cost



- If the cost of every move is 1, is moving the Bishop to safety equivalent to moving Queen?
- Cost losing pieces: Bishop = 3, Queen = 9

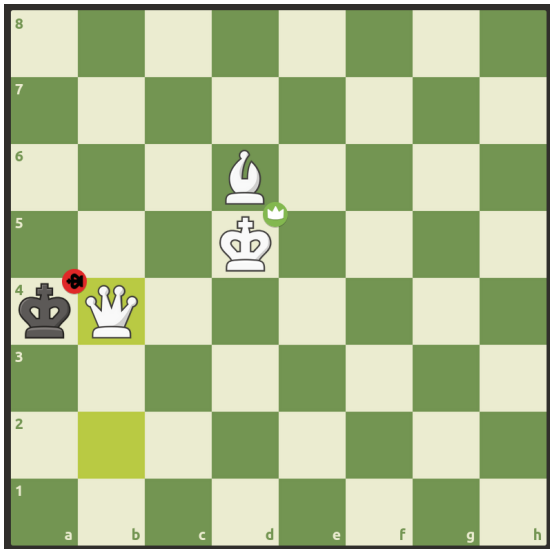
Problem instances: initial and goal states

- Initial state is an element of the state space
- Goal states: all possible states that satisfy a specific condition.



Src: https://lichess.org/editor

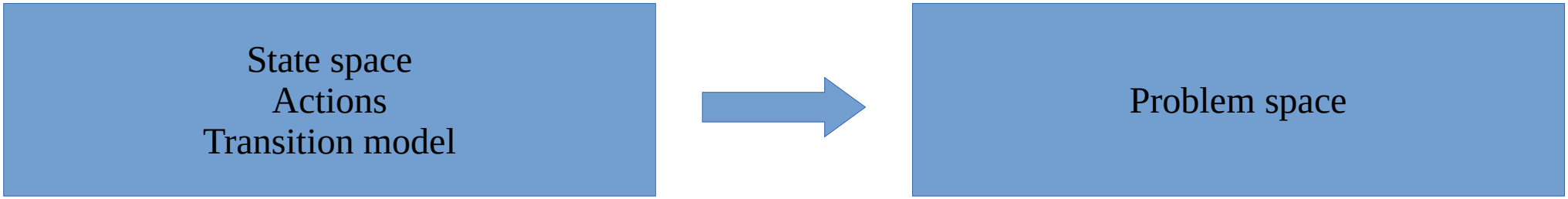




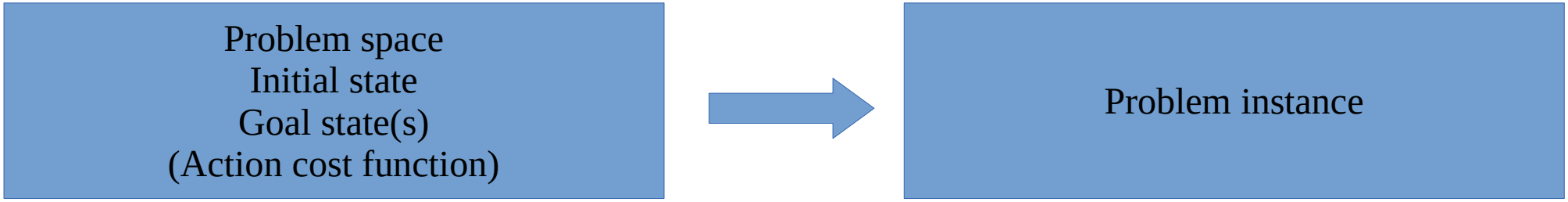
Src: https://www.chess.com/

Problem spaces and instances

- A problem space consists of a set of states of the problem, and a set of operators that change the state



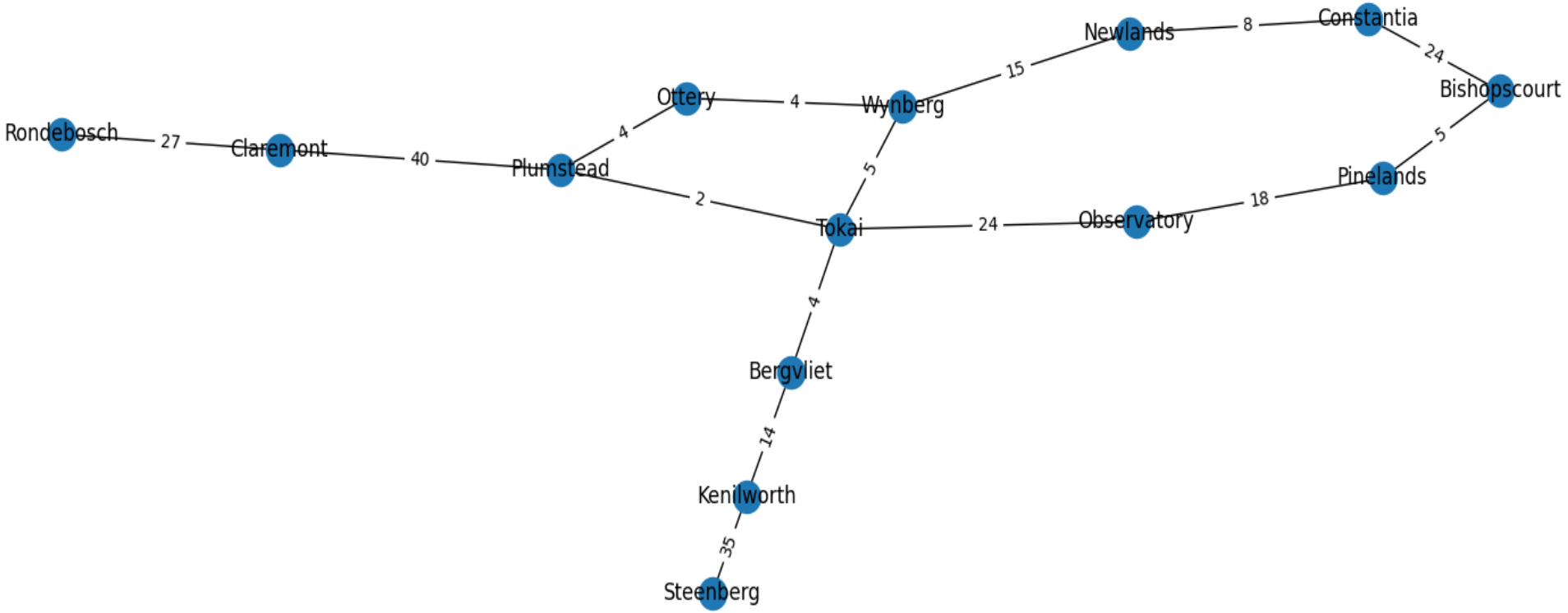
- Problem instance is a problem space together with an initial state and a goal state



- Search task
  - **Path**: Any sequence of actions
  - **Solution**: a path from the initial state to a goal state
  - **Optimal** solution has the lowest path cost among all solutions
- **Problem-solving task**: given a problem instance, find a(n optional) solution
- State space can be represented as a graph
  - Node = state, edge = action.
  - Edges can (un)directed. Depends on reversibility of action.

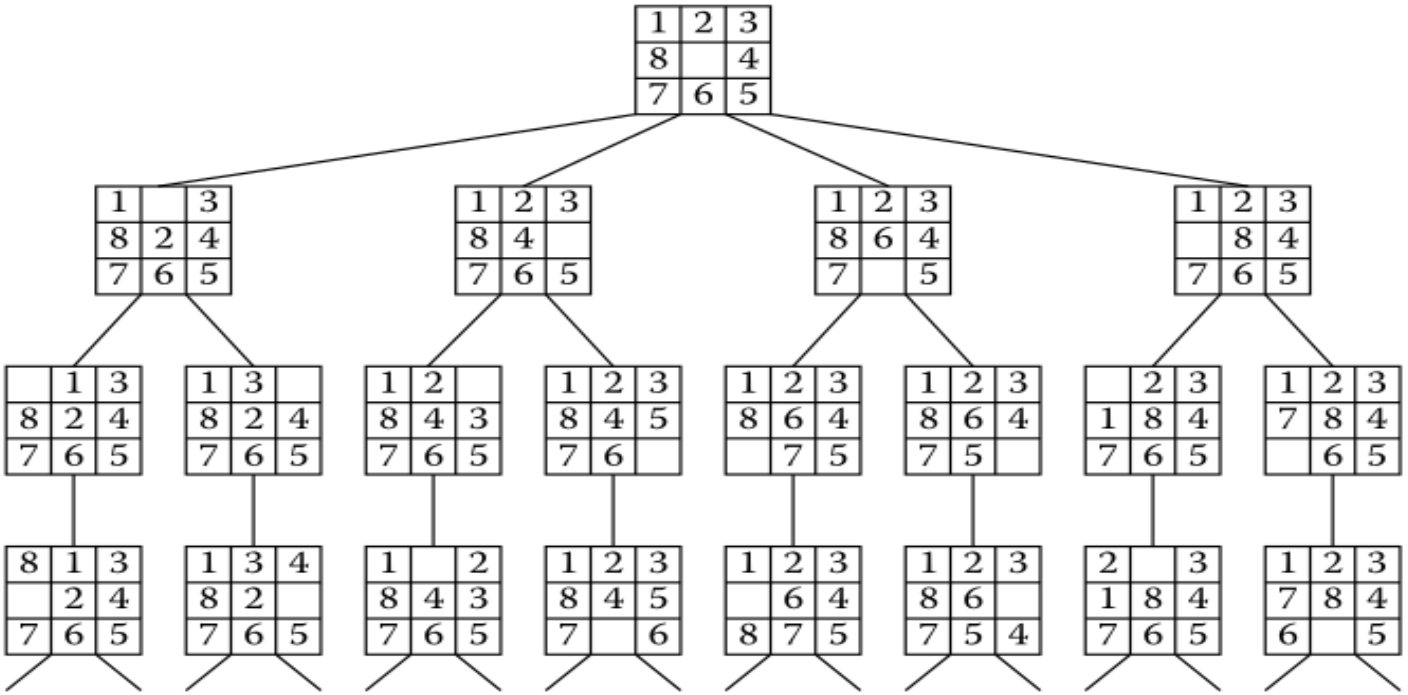
State space representation

- State space can be represented as a graph



State space representation

- State space can be represented as a graph
- For simplicity, use a tree to represent state space
  - the initial state is the root of the tree
  - when state can be reached by different paths, duplicate the node



(Credit: Richard E. Korf)

Tree search vs. graph search

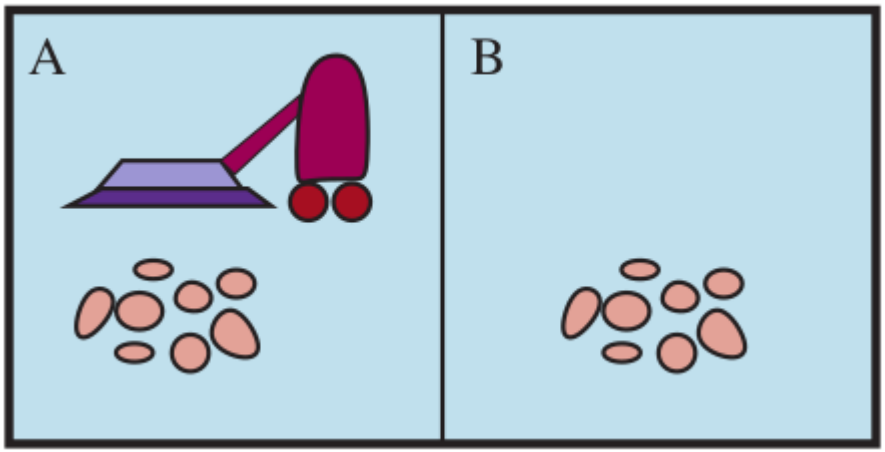
- Tree Search: explores paths without revisiting nodes
- Graph Search: considers all possible paths, including cycles
- Applications: shortest path algorithms

Storage implications

- Size of the graph(s) involved matters
- Consider state-space complexity for tic-tac-toe
  - In a 3×3 grid, each of the 9 cells can be in one of three states: empty, X, or O
  - Number of states $3^9 = 19{,}683$
- Space considerations (naively) for chess:
  - Total possible unique games (Shannon): 10^{120}
  - Using 2 bytes for each node then you need 2×10^{120} bytes
  - In context, value is larger than the number of atoms in the entire observable universe

Example: vacuum world

- There exists a world consisting of two rooms (Square A and Square B).
- Each cell can have dirt, robot
- If there are n cells, total state space is $n*2^n$
- Actions = [clean, L/R/U/D]
- Initial state: any state can act as initial state

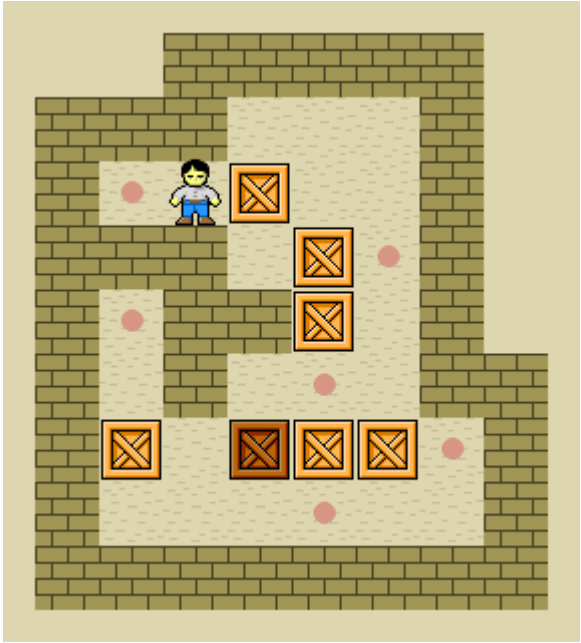


- Goal states = $\{c_{ij}[clean]\}$ for all i,j

- Transition: result(c_{ij}[dirty], clean) = c_{ij}[clean]

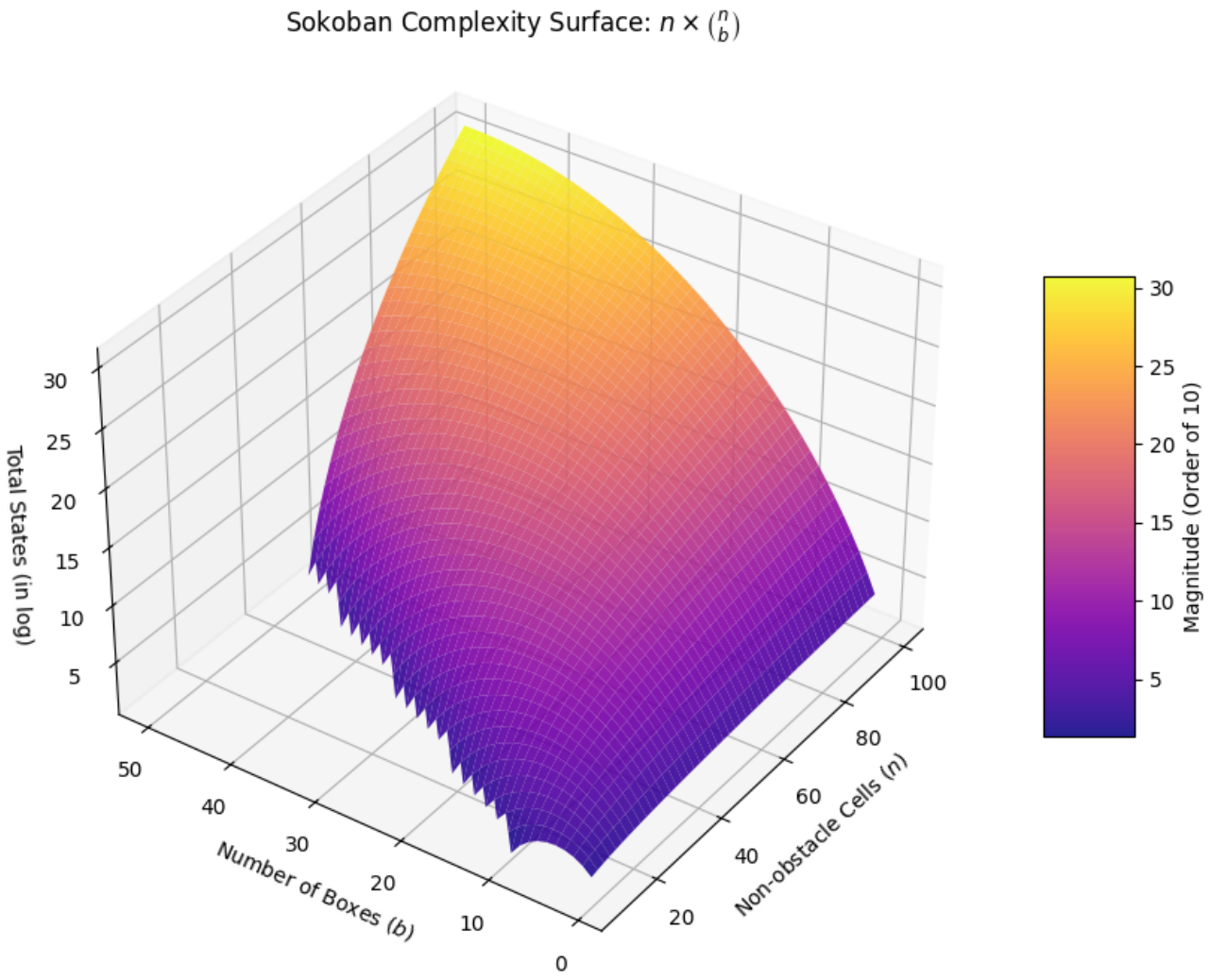
Example: sokoban

- Agent pushes boxes to get them onto specific locations
- Exercise: formulate problem instance for 9x9 grid with 1 box.



Src: wikipedia

Example: sokoban



Reading

- Chapter 3

